SISTEMAS OPERATIVOS AVANZADOS · UDIT — UNIVERSIDAD DE DISEÑO, INNOVACIÓN Y TECNOLOGÍA · CURSO 2025–2026

**v20 — entrega final**

# UDIT PixelForge

Guía de construcción paso a paso — cómo construimos nuestra distribución Linux live personalizada desde cero, qué problemas tuvimos y cómo los resolvimos.

AUTORES

Andrea Ávila Romero · Iván Herrera González

BASE DEL SISTEMA

Debian 13 Trixie ARM64 · libre.debian.net

VERSIÓN

v20 final · 19 mayo 2026 · 1,2 GB

PAQUETES

1660 (dpkg)

REPOSITORIO

github.com/Andreavilaro0/udit-pixelforge

SHA-256

3f8a04a8…612f

TABLA DE CONTENIDOS

# Índice


01 Herramientas y entorno de trabajo

02 Primeras versiones con Cubic (v8–v14)

03 Diagnóstico: por qué Parallels no arrancaba

04 Cambio de estrategia: base libre.debian.net (v16)

05 Personalización del sistema (v16 → v18)

06 Errores post-arranque y cómo los resolvimos

07 Reducción del tamaño: slimming agresivo

08 Ampliación v20: seguridad ofensiva y robótica

09 El menú interactivo: robomenu

10 Sistema final: cómo quedó todo

11 Cómo probar la ISO

12 Lecciones aprendidas

01 — HERRAMIENTAS

# Herramientas y entorno de trabajo

Antes de empezar a construir la ISO necesitamos tener claro qué se usa para cada cosa. Estas son todas las herramientas que utilizamos a lo largo del proyecto, con una explicación de para qué sirve cada una.

// **SOFTWARE PRINCIPAL**

| HERRAMIENTA | PARA QUÉ LA USAMOS | CUÁNDO |
|---|---|---|
| Cubic | Interfaz gráfica para personalizar ISOs Debian/Ubuntu. Abre un entorno chroot dentro de la ISO y permite instalar paquetes sin salir de la aplicación. **Lo abandonamos en la v15** porque genera un efi.img de 32 MB que impide el arranque en Parallels. | v8–v14 |
| unsquashfs | Extrae el `filesystem.squashfs` de la ISO — el archivo comprimido que contiene todo el sistema de archivos del live. Sin él no se puede acceder al interior del sistema para modificarlo. | v15 en adelante |
| mksquashfs | Reempaqueta el sistema de archivos modificado de vuelta en un `filesystem.squashfs`. Es la operación más lenta del pipeline: tarda entre 10 y 20 minutos con zstd nivel 22. | v15 en adelante |
| xorriso | Genera la imagen ISO final a partir de los archivos del live. El comando que usamos incluye flags específicas para Parallels en Apple Silicon — especialmente `-isohybrid-apm-hfsplus`. | v15 en adelante |
| Parallels Desktop | Máquina virtual en Mac Apple Silicon. Usamos dos instancias: una VM con Debian ARM64 donde hacemos el build, y otra VM donde probamos la ISO resultante. | Todo el proyecto |
| QEMU | Emulador más permisivo que Parallels. Lo usamos para pruebas rápidas de boot. Importante: QEMU ignora algunos errores del bootloader que Parallels rechaza — lo que hizo que al principio pensáramos que nuestra ISO funcionaba cuando en realidad no. | Todo el proyecto |
| ImageMagick | Redimensionar y convertir las imágenes del branding. El boot splash pasó de PNG de 7,4 MB a JPG de 186 KB con este comando: `convert splash.png -quality 85 -resize 1920x1200^ -gravity center -extent 1920x1200 splash.jpg` | v16–v18 |
| fzf + chafa | `fzf` es el selector interactivo del menú robomenu. `chafa` convierte los iconos PNG en caracteres Unicode coloreados para renderizarlos en la terminal. | v20 |

## // ENTORNO DE TRABAJO

Todo el trabajo se hizo sobre un Mac con Apple Silicon (M-series). Dentro de Parallels Desktop tenemos una VM de Debian 13 ARM64 que usamos como máquina de build. Desde esa VM hacemos chroot en la ISO y ejecutamos los scripts de build. La carpeta compartida de Parallels ( `/media/psf/Home/` dentro de la VM) nos permite copiar la ISO resultante directamente al Mac sin corrupción.

> **IMPORTANTE — CÓMO COPIAR LA ISO AL MAC**
>
> Si copias la ISO via OneDrive o cualquier sync cloud, el archivo se corrompe. El SHA256 cambia. La forma correcta es usar la carpeta compartida de Parallels: en la VM se monta en `/media/psf/Home/`, y es un acceso directo al home del Mac. Copia la ISO ahí y el hash queda idéntico.

---

02 — PRIMERAS VERSIONES

# Primeras versiones con Cubic (v8–v14)

Al principio buscamos qué herramienta se usaba para personalizar ISOs de Debian y encontramos **Cubic** (Custom Ubuntu ISO Creator). Tiene una interfaz gráfica que abre un chroot directamente en la ISO y permite instalar paquetes sin salir de la aplicación. Nos pareció ideal porque podíamos ver el sistema en tiempo real.

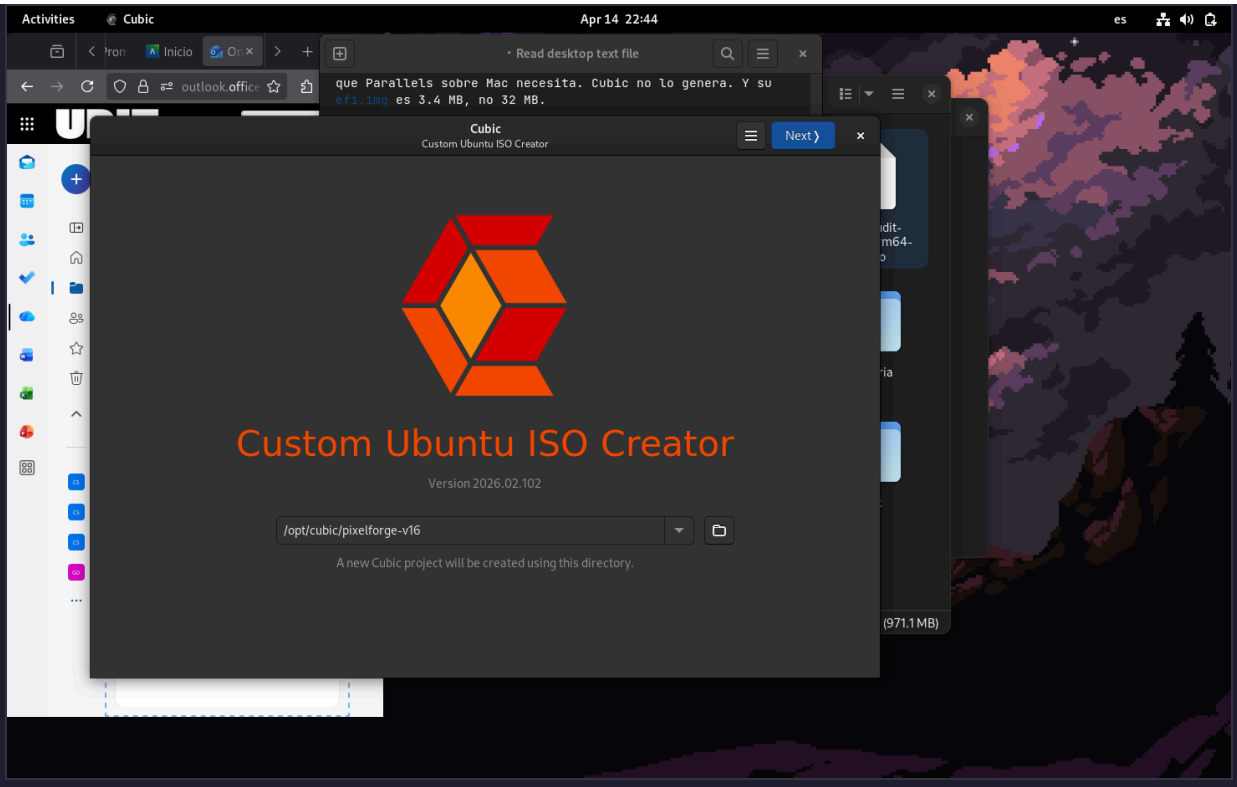
## // CÓMO USAMOS CUBIC

### PASO 1 — INSTALAR CUBIC EN LA VM DE DEBIAN

```
sudo apt-add-repository universe
sudo apt-add-repository ppa:cubic-wizard/release
sudo apt update
sudo apt install cubic
```

### PASO 2 — CARGAR LA ISO BASE Y ENTRAR AL CHROOT

Abrimos Cubic, seleccionamos el directorio del proyecto (`/opt/cubic/pixelforge-v16`) y cargamos la ISO base de Debian 13 Trixie ARM64. Cubic extrae la ISO y abre un terminal dentro del chroot — a partir de ahí todo lo que ejecutamos afecta al sistema que va a quedar en la ISO.




Cubic 2026.02 — directorio del proyecto en /opt/cubic/pixelforge-v16 con la ISO base cargada y el chroot listo para recibir comandos

### PASO 3 — INSTALAR LOS PAQUETES DEL ENUNCIADO DENTRO DEL CHROOT

```
# Herramientas de red
apt-get install -y nmap wireshark tshark tcpdump netcat-openbsd arp-sc

# Seguridad defensiva
apt-get install -y lynis fail2ban ufw chkrootkit rkhunter clamav aide

# Diagnóstico
apt-get install -y htop btop glances lsof iotop sysstat strace fastfet

# Gestión de discos y recuperación
apt-get install -y gparted smartmontools testdisk ddrescue cryptsetup

# WiFi
apt-get install -y iw wavemon wireless-tools network-manager
```

## // RESULTADO: LA V8 ARRANCA EN QEMU PERO NO EN PARALLELS

Con Cubic construimos la v8 (797 MB) y varias versiones más hasta la v14 (945 MB). Al probar en QEMU todo funcionaba: el sistema arrancaba hasta la pantalla de login. Pero cuando lo probamos en Parallels Desktop — nada. Pantalla negra. Ni el GRUB llegaba a aparecer.

> **ERROR — PARALLELS DESKTOP: PANTALLA NEGRA AL ARRANCAR**
>
> Todas las versiones v8–v14 fallaban en Parallels Desktop sobre Mac Apple Silicon. El proceso era siempre igual: logo de Parallels → pantalla negra → nada. Sin mensajes de error, sin log visible. QEMU sí arrancaba.

Al principio pensamos que el problema era algún paquete o la configuración del GRUB. Seguimos construyendo versiones (v9, v10, v11... hasta la v14), añadiendo más herramientas, ajustando configuraciones — pero el problema de arranque en Parallels persistía en todas ellas.

---

03 — DIAGNÓSTICO

# Diagnóstico: por qué Parallels no arrancaba

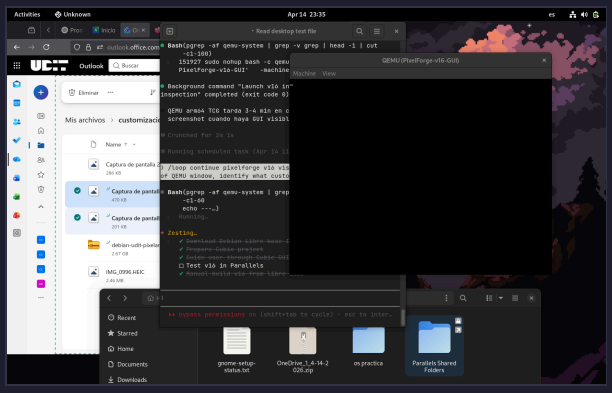
Para entender el problema tuvimos que comparar nuestra ISO con ISOs que sí arrancaban en Parallels. Usamos `xorriso` para leer los metadatos del bootloader de cada ISO y compararlos.

## // HERRAMIENTA DE DIAGNÓSTICO: XORRISO REPORT_EL_TORITO

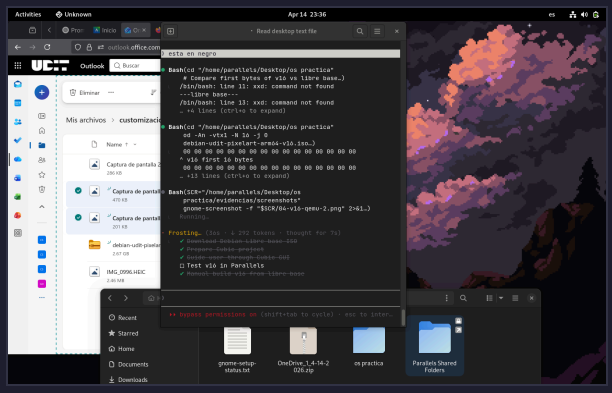
El catálogo El-Torito es la parte de una ISO que le dice al firmware cómo arrancar. El campo `Ldsiz` indica cuántos bloques de 512 bytes tiene que cargar el firmware para poder iniciar. Si ese valor es cero, el firmware no sabe qué cargar.

```
# Comando para ver el catálogo El-Torito de una ISO
xorriso -indev debian-udit-pixelart-arm64-v14.iso -report_el_torito cmd
```

## // LO QUE ENCONTRAMOS: LDSIZ = 0



Comparando el catálogo El-Torito de la ISO de Cubic (arriba) con la de libre.debian.net (abajo). La diferencia en Ldsiz es el problema.



Output completo del análisis — Ldsiz=0 en nuestra ISO de Cubic, Ldsiz=6848 en la base libre

Nuestra ISO mostraba esto:

```
El Torito boot img : 1 UEFI y none 0x0000 0x00 0 63
```

La ISO oficial de libre.debian.net que sí arrancaba mostraba esto:

```
El Torito boot img : 1 UEFI y none 0x0000 0x00 6848 63
```

**CAUSA RAÍZ — EL CAMPO LDSIZ EN EL CATÁLOGO EL-TORITO VALE 0**

Cubic genera un `efi.img` de exactamente **32 MB**. El campo `Ldsiz` del catálogo El-Torito es de 16 bits — puede almacenar como máximo 65535 bloques de 512 bytes, que son exactamente 32 MB. Cuando Cubic intenta registrar 32 MB en un campo de 16 bits, el valor se desborda y queda en `0`.

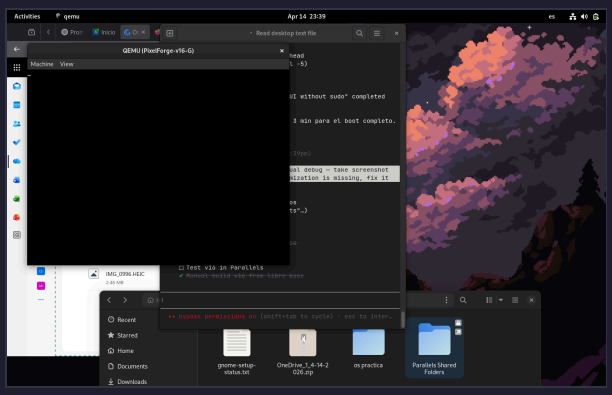
**Parallels Desktop** sobre Apple Silicon interpreta `Ldsiz=0` como "no sé cuánto cargar" y rechaza el arranque silenciosamente.

**QEMU** ignora ese valor y arranca igualmente — por eso nuestra ISO funcionaba en QEMU pero no en Parallels.

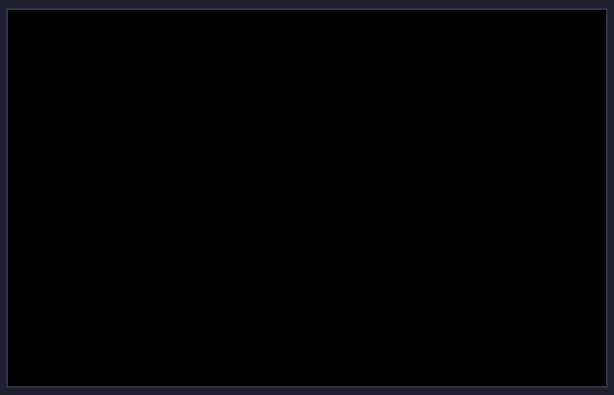
## // POR QUÉ NO SE PUEDE ARREGLAR CON CUBIC

El `efi.img` de 32 MB lo genera Cubic automáticamente y no hay opción de cambiarlo. Además, le falta otro flag crítico en el comando de xorriso que usa: `-isohybrid-apm-hfsplus`. Este flag añade un **Apple Partition Map** a la ISO, que Parallels necesita para arrancar en Apple Silicon. Cubic nunca lo incluye.

La conclusión fue clara: **hay que abandonar Cubic y construir la ISO manualmente**.



QEMU con la ISO arrancada — el sistema funcionaba aquí, lo que confundió el diagnóstico durante días



Terminal con el análisis completo — comparando los dos campos clave: Ldsiz y el flag APM

04 — CAMBIO DE ESTRATEGIA

# Cambio de estrategia: base libre.debian.net (v16)

La solución fue cambiar la ISO base. **libre.debian.net** es el repositorio de Debian que publica live builds ARM64 — la web oficial de debian.org solo tiene live builds para amd64. La ISO de libre.debian.net tiene exactamente lo que necesitamos: un `efi.img` pequeño (3,4 MB) y el flag `-isohybrid-apm-hfsplus`.

### // PASO 1: DESCARGAR LA ISO BASE

```
# Descargar desde libre.debian.net
wget https://cdimage.debian.org/cdimage/unofficial/free/cd-including-firmware
```

### // PASO 2: EXTRAER EL COMANDO XORRISO DE LA PROPIA ISO

Debian guarda dentro de cada ISO el comando exacto que se usó para generarla, en `.disk/mkisofs`. Esto nos dio el comando correcto sin necesidad de buscarlo en ningún sitio:

```
# Montar la ISO y leer el comando de generación
mkdir -p /mnt/iso
mount -o loop debian-live-13.3.0-arm64-libre-xfce.iso /mnt/iso
cat /mnt/iso/.disk/mkisofs
```

El output mostraba el flag que necesitábamos:

```
xorriso -as mkisofs -R -r -J -joliet-long -l -cache-inodes -iso-level 3 \
  -V "d-libre-13.3.0-xfce-arm64" \
  -e boot/grub/efi.img -no-emul-boot \
  -isohybrid-gpt-basdat \
  -isohybrid-apm-hfsplus \
  -append_partition 2 0xef boot/grub/efi.img \
  -o out.iso binary/
```

### // PASO 3: EXTRAER EL SQUASHFS

```
mkdir -p work/iso work/rootfs
# Copiar el árbol de la ISO (excluyendo el squashfs)
rsync -av --exclude=filesystem.squashfs /mnt/iso/ work/iso/

# Extraer el sistema de archivos
unsquashfs -d work/rootfs /mnt/iso/live/filesystem.squashfs
```

## // PASO 4: HACER CHROOT E INSTALAR PAQUETES

```
# Preparar el entorno chroot
mount --bind /dev  work/rootfs/dev
mount --bind /proc work/rootfs/proc
mount --bind /sys  work/rootfs/sys
cp /etc/resolv.conf work/rootfs/etc/resolv.conf

# Entrar al chroot
chroot work/rootfs /bin/bash

# Dentro del chroot:
apt-get update
apt-get install -y [lista de paquetes]
```

## // PASO 5: REEMPAQUETAR CON ZSTD Y GENERAR LA ISO

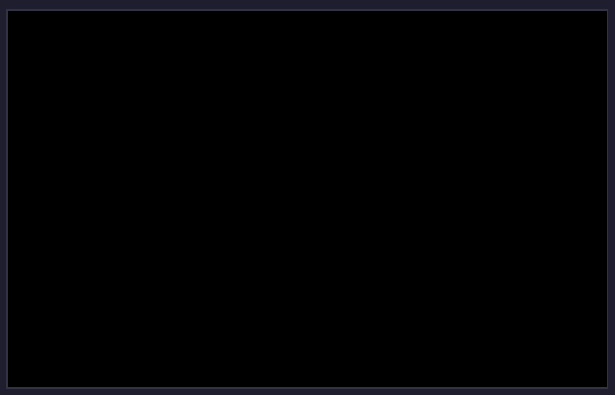
```
# Salir del chroot y desmontar
exit
umount work/rootfs/{dev,proc,sys}

# Reempaquetar el sistema de archivos
mksquashfs work/rootfs work/iso/live/filesystem.squashfs \
  -comp zstd -Xcompression-level 22 -b 1M \
  -processors 2 -noappend

# Generar la ISO final con el comando de libre.debian.net
xorriso -as mkisofs -R -r -J -joliet-long -l -cache-inodes -iso-level 3 \
  -V "DEBIAN_UDIT_EDITION" \
  -e boot/grub/efi.img -no-emul-boot \
  -isohybrid-gpt-basdat \
  -isohybrid-apm-hfsplus \
  -append_partition 2 0xef boot/grub/efi.img \
  -o debian-udit-v16.iso work/iso/
```

**RESULTADO — LA V16 ARRANCA EN PARALLELS POR PRIMERA VEZ**

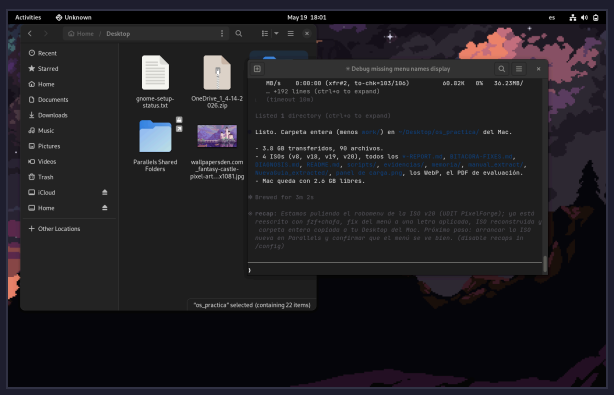
Verificamos el catálogo El-Torito de la nueva ISO:

```
El Torito boot img : 1 UEFI y none 0x0000 0x00 6848 63
```

`Ldsiz=6848` — valor válido. La ISO arrancó en Parallels Desktop después de 6 días trabajando en el problema del bootloader.



v16 con la base libre.debian.net — squashfs zstd correcto, ISO con APM flag, arrancando en QEMU



La VM de build (Debian ARM64) corriendo dentro de Parallels — aquí es donde ejecutamos todos los scripts

05 — PERSONALIZACIÓN

# Personalización del sistema

Una vez que la ISO arrancaba correctamente en Parallels, empezamos a aplicar toda la personalización visual: tema, wallpapers, Plymouth, usuarios y panel de XFCE.

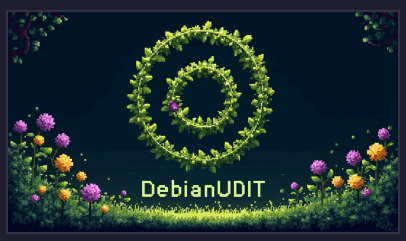
## // TEMA CATPPUCCIN MOCHA BLUE

El sistema usa el tema **Catppuccin Mocha Blue** — un esquema de color oscuro con tonos azules como acento principal. Es el tema que veréis en este propio documento. Lo instalamos desde el repositorio oficial de Catppuccin para GTK:

```
cd /tmp
git clone --depth=1 https://github.com/catppuccin/gtk catppuccin-gtk
cp -r catppuccin-gtk/themes/catppuccin-mocha-blue-standard+default \
      /usr/share/themes/
# Activar como tema del sistema
gsettings set org.gnome.desktop.interface gtk-theme \
          "catppuccin-mocha-blue-standard+default"
```

## // WALLPAPERS

El sistema tiene tres imágenes de fondo distintas, una para cada momento del arranque:




**Plymouth (boot splash)** — portal en bosque pixel art. Aparece mientras carga el kernel, antes de llegar al login.

**LightDM (login)** — imagen del hotel con los avatares de Andrea e Iván. El texto "Andrea e Iván crearon esto :)" está en el wallpaper del escritorio.

**Escritorio XFCE** — edificio UDIT en pixel art. El wallpaper del sistema operativo en funcionamiento.

**ERROR — IMÁGENES EN LUGARES EQUIVOCADOS**

Durante las primeras versiones las tres imágenes estaban asignadas al lugar incorrecto. El portal aparecía en el escritorio, el hotel en el boot splash. No lo detectamos hasta que tomamos capturas reales del sistema funcionando y las comparamos.

El problema era que las configurábamos en el chroot sin poder ver cómo quedaban realmente hasta generar la ISO completa y arrancarla.

**FIX**

Establecer explícitamente qué imagen va a cada lugar:

```
# Plymouth boot splash
cp portal.jpg /usr/share/plymouth/themes/pixelforge/background.jpg
update-initramfs -u  # regenerar siempre después de cambiar Plymouth

# LightDM greeter
cp hotel.jpg /usr/share/backgrounds/pixelforge/login.jpg
# En /etc/lightdm/lightdm-gtk-greeter.conf:
# background = /usr/share/backgrounds/pixelforge/login.jpg

# XFCE desktop wallpaper
cp udit-building.jpg /usr/share/backgrounds/pixelforge/wallpaper.jpg
```

## // PLYMOUTH — BOOT SPLASH

Plymouth es el programa que muestra la pantalla de carga mientras arranca el kernel. El tema que creamos muestra la imagen del portal en pixel art a pantalla completa, con una barra de progreso en la parte inferior.

progress_box.png — el contenedor de la barra de progreso en el boot splash

progress_bar.png — la barra de progreso que avanza mientras carga el sistema

**ERROR — PLYMOUTH NO MOSTRABA LA IMAGEN**

En la primera versión pusimos el PNG original del portal como boot splash. El archivo tenía 7,4 MB. El problema es que Plymouth embebe las imágenes del tema dentro del initramfs — si el archivo es muy grande, no se embebe correctamente.

**FIX**

```
# Convertir el PNG a JPG comprimido (7.4 MB → 186 KB)
convert portal.png -quality 85 -resize 1920x1200^ \
        -gravity center -extent 1920x1200 background.jpg

# Regenerar el initramfs para que incluya el nuevo archivo
update-initramfs -u

# Copiar el initrd actualizado al directorio live de la ISO
cp /boot/initrd.img-$(uname -r) /iso/live/initrd.img
```

## // USUARIOS DEL SISTEMA

```
# Crear los usuarios
useradd -m -s /bin/bash danielsson
useradd -m -s /bin/bash user
useradd -m -s /bin/bash admin

# Establecer contraseñas
echo "danielsson:root" | chpasswd
echo "user:live"       | chpasswd
echo "admin:admin"     | chpasswd
echo "root:root"       | chpasswd

# Añadir a los grupos necesarios
usermod -aG sudo,wireshark,audio,video,netdev,plugdev danielsson
usermod -aG sudo,wireshark,audio,video,netdev,plugdev user
usermod -aG sudo,wireshark admin

# Configurar autologin en LightDM
# En /etc/lightdm/lightdm.conf:
# autologin-user = danielsson
# autologin-user-timeout = 0
```

**ERROR — LAS CONTRASEÑAS NO FUNCIONABAN DESPUÉS DE REEMPAQUETAR**

Varias veces las contraseñas que configurábamos en el chroot dejaban de funcionar en la ISO final. El problema era que `chpasswd` se ejecutaba, pero después en el mismo chroot había operaciones que restauraban el `/etc/shadow` original.

**FIX**

Verificar el contenido de `/etc/shadow` justo antes de ejecutar `mksquashfs` para confirmar que los hashes de contraseña son los correctos:

```
grep -E "^(danielsson|user|admin|root):" /etc/shadow
# Comprobar que los hashes no son '*' ni '!' (que significan cuenta bloq
```

---

06 — ERRORES POST-ARRANQUE

# Errores post-arranque y cómo los resolvimos

Una vez que la ISO arrancaba en Parallels, aparecieron varios problemas de funcionamiento del escritorio. Los listamos todos con su causa y solución.

## // ERROR 1: XFCE SE TRABABA AL INICIAR SESIÓN

El sistema arrancaba y llegaba al login, pero al entrar la sesión se congelaba. El escritorio no cargaba.

**CAUSAS ENCONTRADAS (ERAN CINCO SIMULTÁNEAS)**

1. `xfce4-session` **estaba eliminado** — un `apt autoremove` anterior lo había quitado porque estaba marcado como "Recommended" en lugar de "Required". Sin él XFCE no puede iniciar una sesión.
2. `light-locker` **activo** — este programa de bloqueo de pantalla se activaba justo al entrar al escritorio y mostraba un prompt de contraseña encima, lo que parecía que el sistema se había colgado.
3. `calamares` **en el autostart** — el instalador de Debian arrancaba automáticamente al iniciar sesión.
4. **Dos agentes polkit en conflicto** — tanto `polkit-mate` como `polkit-ukui` estaban en el autostart y competían entre sí.
5. **El panel de XFCE referenciaba .desktop files inexistentes** — crasheaba al intentar cargar los launchers.

**FIX**

```
# Reinstalar xfce4-session explícitamente
apt-get install -y xfce4-session

# Eliminar light-locker
apt-get purge -y light-locker

# Limpiar autostart de elementos problemáticos
rm -f /etc/xdg/autostart/calamares-desktop-icon.desktop
rm -f /etc/xdg/autostart/light-locker.desktop
rm -f /etc/xdg/autostart/polkit-ukui-authentication-agent-1.desktop
rm -f /etc/xdg/autostart/orca-autostart.desktop
rm -f /etc/xdg/autostart/im-launch.desktop
```

### // ERROR 2: LA CAJA DE LOGIN TAPABA EL TEXTO DEL WALLPAPER

El fondo del login (LightDM) tiene el texto "Andrea e Iván crearon esto :)" en la parte central de la imagen. Con la caja de login centrada verticalmente, el formulario tapaba exactamente ese texto.

**FIX — MOVER LA CAJA AL 78% DE LA ALTURA**

```
# En /etc/lightdm/lightdm-gtk-greeter.conf:
[greeter]
background = /usr/share/backgrounds/pixelforge/login.jpg
position = 50%,center 78%,center    # caja al 78% vertical
```

### // ERROR 3: RESOLUCIÓN MÍNIMA 1024×768

Sin las Parallels Tools instaladas, Parallels asigna una resolución de 1024×768 por defecto. Todos los wallpapers y el escritorio se veían cortados o con aspecto distorsionado.

**FIX — FORZAR RESOLUCIÓN DESDE EL KERNEL Y XFCE**

```
# 1. Añadir al cmdline del kernel en /etc/default/grub:
GRUB_CMDLINE_LINUX="... video=1920x1080"
update-grub

# 2. Script de autostart que fuerza xrandr al inicio de sesión:
# En /etc/xdg/autostart/pixelforge-xrandr.desktop
[Desktop Entry]
Type=Application
Exec=xrandr --output Virtual-1 --mode 1920x1080
X-GNOME-Autostart-enabled=true
```

## // ERROR 4: NOMBRE EQUIVOCADO EN LA PANTALLA DE LOGIN

El sistema mostraba "Daniel Baumann" en la pantalla de login en lugar del nombre de usuario. Esto ocurrió porque el usuario `danielsson` se creó con el campo GECOS del usuario original de la ISO base (el creador de Debian Live se llama Daniel Baumann).

**FIX**

```
# Limpiar el campo GECOS (nombre completo) del usuario
usermod -c "" danielsson
# O directamente en /etc/passwd cambiar:
# danielsson:x:1002:1002:Daniel Baumann (Debian Live):/home/danielsson:/
# por:
# danielsson:x:1002:1002::/home/danielsson:/bin/bash
```

## // ERROR 5: EL WALLPAPER APARECÍA REPETIDO (TILING)

La imagen del edificio UDIT tenía dimensiones 1080×607 — aspect ratio de 16:9. La pantalla de Parallels es 1920×1200, que es 16:10. La diferencia hacía que XFCE aplicara tiling (repetición) en lugar de escalar.

**FIX — REDIMENSIONAR CON LETTERBOXING**

```
# Redimensionar con ImageMagick a 1920x1200 con barras negras (letterbox
convert walpaper.webp -resize 1920x1200^ \
        -gravity center -extent 1920x1200 wallpaper.jpg

# En la configuración de XFCE: image-style = 5 (zoom, sin repetición)
```

## // ERROR 6: SHA256 DIFERENTE AL COPIAR LA ISO AL MAC

Esto nos costó bastante tiempo. Generábamos la ISO en la VM, la copiábamos a OneDrive para descargarla en el Mac, y el SHA256 cambiaba. Pensábamos que la ISO estaba corrupta.

**CAUSA**

OneDrive y otros sync cloud pueden alterar archivos binarios grandes durante la transferencia — en particular los ISOs que contienen padding de ceros. El archivo llegaba diferente al destino.

**FIX — USAR LA CARPETA COMPARTIDA DE PARALLELS**

Activar "Share Mac folders" en la configuración de la VM (Configure → Options → Sharing). Dentro de la VM aparece `/media/psf/Home/` que es acceso directo al home del Mac. Copiar la ISO ahí da siempre SHA256 idéntico.

```
# Dentro de la VM:
cp debian-udit-v18.iso /media/psf/Home/Downloads/
# En el Mac: el archivo está en ~/Downloads/ con el hash correcto
```

07 — REDUCCIÓN DE TAMAÑO

# Reducción del tamaño: slimming agresivo

La ISO de libre.debian.net completa pesaba **2,8 GB**. Para que fuera usable como live USB necesitábamos reducirla significativamente. Lo hicimos en varias rondas de purga de paquetes.

### // RONDA 1 — ELIMINAR LO MÁS OBVIO (2,8 GB → 2,0 GB)

```
apt-get purge -y firefox-esr firefox-esr-l10n-* \
              libreoffice* \
              texlive* \
              'man-db' 'manpages*'
apt-get autoremove -y
apt-get clean
```

### // RONDA 2 — FUENTES, MULTIMEDIA PESADA (2,0 GB → 1,2 GB)

```
# Fuentes asiáticas (japonés, chino, coreano) — enormes y no necesarias
apt-get purge -y 'fonts-noto-cjk*' 'fonts-noto-cjk-extra*'

# Motor de WebKit (para apps GTK que no necesitamos)
apt-get purge -y libwebkit2gtk-4.0* libwebkit2gtk-4.1*

# Qt WebEngine
apt-get purge -y 'libqt5webengine*' 'qtwebengine*'

# Librerías de datos geoespaciales (no relevantes para el proyecto)
apt-get purge -y libgdal*

# Hardware acceleration de Intel (no aplica en ARM)
apt-get purge -y libmfx1 libvpl2 libvpx7
```

### // RONDA 3 — SLIM AGRESIVO FINAL (1,2 GB → 883 MB)

```
  apt-get purge -y ibus ibus-data \
                   mesa-vulkan-drivers \
                   espeak-ng espeak-ng-data \
                   speech-dispatcher* \
                   samba-libs samba-common \
                   cpp-14 gcc-14 libgcc-14-dev \
                   libgs10 libgs-common \
                   'hunspell-*' mythes-* \
                   anthy-common \
                   libdnnl3 \
                   libz3-4 libz3-dev
  apt-get autoremove --purge -y
```

// **EVOLUCIÓN DEL TAMAÑO**

| VERSIÓN | FECHA | TAMAÑO | QUÉ CAMBIÓ |
|---|---|---|---|
| v8 | 12 abr | 797 MB | Primera entrega con Cubic |
| v14 | 12 abr 15:35 | 945 MB | Última iteración Cubic, más paquetes instalados |
| v16 base | 15 abr 00:03 | 2,8 GB | Cambio a libre.debian.net sin modificar |
| v16 slim-1 | 15 abr 07:52 | 2,0 GB | −Firefox, −LibreOffice, −documentación |
| v16 slim-2 | 15 abr 10:26 | 1,2 GB | −Fuentes CJK, −Qt WebEngine, −libwebkit |
| v17 | 15 abr 12:34 | 1,2 GB | Wallpapers correctos, autologin danielsson |
| **v18** | 15 abr 22:10 | **883 MB** | Slim agresivo final + login pulido — primera entrega oficial |
| v19 | mayo | 989 MB | Refinamientos menores, base para v20 |
| **v20** | 19 may 2026 | **1,2 GB** | Entrega final — seguridad ofensiva + robótica + robomenu |

08 — AMPLIACIÓN V20

# Ampliación v20: seguridad ofensiva y robótica

El 27 de abril de 2026 el profesor publicó una nueva guía ("RoboSecOS") que pedía añadir herramientas de seguridad ofensiva, robótica/hardware y un menú interactivo. La guía estaba pensada para Ubuntu 24.04 + Cubic, pero el profesor confirmó que se aceptaba una adaptación sobre nuestra base.

Nuestra decisión fue **añadir encima de la v19** sin tocar el branding ni los usuarios existentes. Cambiar el usuario `danielsson` por `robosec` o el autologin de XFCE a TTY hubiera roto todo lo que ya estaba verificado funcionando.

## // PAQUETES NUEVOS QUE NECESITÁBAMOS INSTALAR

```
# Seguridad ofensiva
masscan hydra john hashcat aircrack-ng gobuster whois

# Robótica y hardware
avrdude minicom picocom screen python3-pip

# Herramienta Espressif para ESP32/ESP8266
pip3 install esptool
```

## // ERROR 1: APT CANCELA TODO SI UN PAQUETE NO EXISTE

Nuestro primer intento fue instalar todos los paquetes en una sola llamada:

```
apt-get install -y masscan nikto hydra john hashcat aircrack-ng gobuster whoi
```

Resultado:

```
E: Unable to locate package nikto
```

**ERROR — APT CANCELA TODA LA TRANSACCIÓN SI UN PAQUETE NO EXISTE**

`nikto` fue eliminado de Debian 13 Trixie (estaba en Debian 12 bookworm pero no llegó a Trixie). Cuando apt no encuentra un paquete, cancela toda la transacción — no instala ninguno de los demás.

Resultado: la primera pasada no instaló absolutamente nada.

**FIX — INSTALAR PAQUETE A PAQUETE CON SET +E**

```
set +e   # no abortar el script si un comando falla
for pkg in masscan hydra john hashcat aircrack-ng gobuster \
           whois avrdude minicom picocom screen python3-pip; do
  echo "==> Instalando: $pkg"
  apt-get install -y "$pkg"
done
set -e   # volver al comportamiento normal
pip3 install esptool
```

## // ERROR 2: APT-LISTS VACÍAS ENTRE SESIONES DE CHROOT

En la segunda sesión de chroot ejecutábamos `apt-get install` y fallaba con "Unable to locate package" para todo — incluso paquetes que sí existen en Debian 13.

**CAUSA — /VAR/LIB/APT/LISTS/ VACÍO**

En la primera sesión habíamos ejecutado `apt-get clean` al final como buena práctica para reducir el tamaño. Esto eliminó también los índices de paquetes (`/var/lib/apt/lists/`). Sin esos índices, apt no sabe qué paquetes existen.

**FIX**

Ejecutar `apt-get update` al inicio de cada sesión de chroot, sin excepciones:

```
# Primera línea de cualquier sesión de chroot que use apt
apt-get update
```

### // ERROR 3 (CRÍTICO): KERNEL PANIC AL ARRANCAR LA V20

El primer ISO v20 generado causaba un kernel panic durante el arranque. El sistema no llegaba ni al GRUB:

```
SQUASHFS error: xz decompression failed, data probably corrupt
SQUASHFS error: Failed to read block 0xf8dda3c: -5
/sbin/init: error while loading shared libraries:
  /lib/aarch64-linux-gnu/libcrypto.so.3: cannot read file data: Input/output
Kernel panic - not syncing: Attempted to kill init! exitcode=0x00007f00
```

Lo extraño: si hacíamos `unsquashfs` sobre el squashfs problemático, los archivos se extraían perfectamente. Los SHA256 de los binarios eran idénticos a los de la v19. El archivo no estaba corrupto desde el punto de vista del userland.

**CAUSA RAÍZ — FILTRO BCJ-ARM INCOMPATIBLE CON KERNEL ARM64**

En nuestro script de build había heredado este comando de la era Cubic:

```
mksquashfs rootfs filesystem.squashfs -comp xz -Xbcj arm -b 128K
```

El flag `-Xbcj arm` activa el filtro BCJ (Branch/Call/Jump) para ARM. Este filtro está diseñado para ARM 32-bit. Cuando se aplica a binarios ARM64, produce un squashfs que:

- `unsquashfs` desde userland puede descomprimir (tiene tolerancia a errores)
- El driver del kernel del kernel 6.12 arm64 **rechaza** al leer bloques durante el arranque

La v19 ya usaba zstd-22 correctamente (verificado con `unsquashfs -s`) pero el script antiguo de build no se había actualizado.

**FIX — RECOMPILAR CON ZSTD, SIN BCJ**

```
mksquashfs work/rootfs work/iso/live/filesystem.squashfs \
  -comp zstd -Xcompression-level 22 -b 1M \
  -processors 2 -noappend
```

Este comando tarda 15–20 minutos en 2 cores, pero produce un squashfs que el kernel ARM64 lee correctamente.

Verificar siempre el formato del squashfs antes de recompilar:

```
unsquashfs -s filesystem.squashfs
# Debe mostrar: Compression zstd, Block size 1048576
```

## // ERROR 4: LOLCAT NO ENCONTRADO EN EL BANNER DE LA TTY

El script del banner del sistema ( `robosec-banner.sh` ) usaba `lolcat` para colorear la salida de figlet. En la v20 el banner fallaba sin mostrar ningún color.

**CAUSA**

Debian instala `lolcat` en `/usr/games/` (convención para herramientas "no serias" como fortune, cowsay, etc.), no en `/usr/bin/` . El script usaba `command -v lolcat` que no encontraba el binario si `/usr/games` no estaba en el PATH.

**FIX**

```
# En robosec-banner.sh, buscar lolcat en múltiples rutas:
LOLCAT=""
for p in /usr/games/lolcat /usr/bin/lolcat /usr/local/bin/lolcat; do
  [ -x "$p" ] && LOLCAT="$p" && break
done

if [ -n "$LOLCAT" ]; then
  figlet -f slant "PixelForge" | $LOLCAT
else
  figlet -f slant "PixelForge"   # degradar a figlet sin color
fi
```

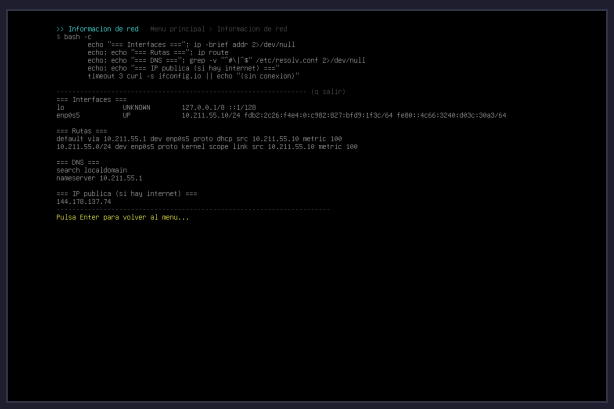
09 — ROBOMENU

# El menú interactivo: robomenu

El menú interactivo era uno de los requisitos de la guía actualizada. El enunciado pedía `whiptail`, pero decidimos ir más lejos: reescribimos el menú con `fzf` como selector y `chafa` para mostrar iconos pixel art en la terminal. El resultado se llama `robomenu`.

Para usarlo: desde cualquier terminal del sistema live, escribir `robomenu` y pulsar Enter. Flechas para navegar, Enter para seleccionar, ESC para volver al nivel anterior.

robomenu — menú principal. El icono pixel art se renderiza con chafa arriba del selector fzf. En TTYs de más de 140 columnas el icono aparece a la izquierda del menú.




Resultado de ejecutar "Ver mi conexión a internet" desde robomenu — muestra IPs, rutas y servidores DNS del sistema en tiempo real.

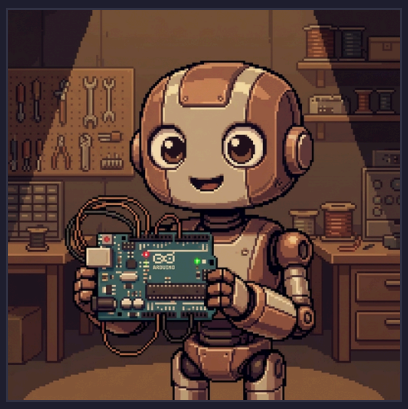
## // ESTRUCTURA DEL MENÚ

| SUBMENÚ | ICONO | HERRAMIENTAS INCLUIDAS |
|---|---|---|
| Seguridad y Redes | `seguridad.png` | nmap, masscan, tcpdump, hydra, john, hashcat, aircrack-ng, gobuster, lynis, fail2ban, información de red |
| Sistema y Diagnóstico | `red.png` | htop/btop, lsof, strace, smartmontools, gparted, testdisk, ddrescue, chntpw, información del sistema |
| Robótica y Hardware | `robotica.png` | avrdude, minicom, esptool, picocom, información de puertos serie disponibles |

## // ICONOS PIXEL ART

Cada submenú tiene su propio icono guardado en `/usr/share/pixelforge/icons/` . Se renderizan con `chafa --size=32x12` antes de lanzar fzf:

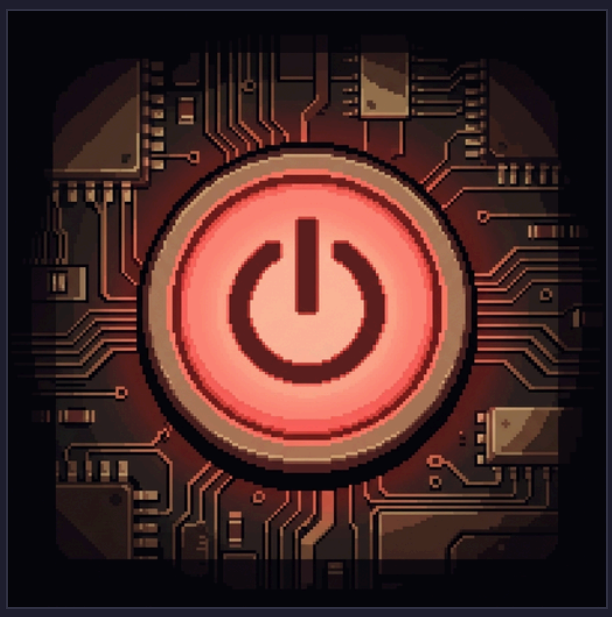
seguridad.png — submenú de seguridad y redes



robotica.png — submenú de robótica y hardware

red.png — submenú de sistema y diagnóstico

inicio.png — icono de la pantalla de inicio del menú



apagar.png — opción de apagar/reiniciar el sistema desde el menú

## // LOS HELPERS RUN Y RUN_PAGER

Cada opción del menú llama a uno de estos dos helpers. Envuelven el comando con información contextual: qué submenú estamos usando, el comando exacto que se ejecuta, la salida, el tiempo que tardó y si terminó bien:

```
>> Escanear red local   Menú principal > Seguridad y Redes
$ nmap -sV --open 192.168.1.0/24
──────────────────────────────────────────────────────────────
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1
Host is up (0.0012s latency).
PORT     STATE SERVICE  VERSION
80/tcp   open  http     lighttpd 1.4.x
443/tcp  open  https    OpenSSL
──────────────────────────────────────────────────────────────
[ok]  (12s)

Pulsa Enter para volver al menú...
```

### // ERROR: EL MENÚ MOSTRABA SOLO UNA LETRA POR OPCIÓN

Al probarlo en Parallels el menú aparecía así:

```
s
r
r
s
a
```

En lugar de los nombres completos de las opciones.

**CAUSA — PREVIEW LATERAL DE FZF EN TTY DE 80 COLUMNAS**

fzf muestra el icono pixel art en un panel de preview. Configuramos ese panel al 55% del ancho de la terminal. En una TTY de 80 columnas (el tamaño por defecto de Parallels), el 55% del ancho son 44 caracteres para el preview — y solo quedan 36 caracteres para el menú. fzf trunca cada opción a lo que cabe en el espacio disponible, que resultó ser exactamente una letra.

**FIX — DETECCIÓN DEL ANCHO Y UMBRAL DE 140 COLUMNAS**

```
# Detectar el ancho real de la terminal con tres fuentes
W1="${COLUMNS:-0}"
W2=$(stty size 2>/dev/null | awk '{print $2}')
W3=$(tput cols 2>/dev/null)
# Tomar el menor valor no-cero
COLS=$(printf '%s\n' "$W1" "$W2" "$W3" | grep -v '^0$' | sort -n | head

# Según el ancho, cambiar la posición del preview
if [ "${COLS:-0}" -ge 140 ]; then
  PREVIEW_POS="right:40%"
else
  PREVIEW_POS="up:40%"   # preview arriba — el menú usa todo el ancho
fi

fzf --preview="..." --preview-window="$PREVIEW_POS"
```

## // EL BANNER DE LA TTY

Al abrir cualquier TTY (Ctrl+Alt+F2 desde XFCE, o arrancar en modo Terminal desde el GRUB), aparece el banner generado por `robosec-banner.sh`:

```
  ____  _          _ _____
 |  _ \(_)_  _____| |  ___|__  _ __ __ _  ___
 | |_) | \ \/ / _ \ | |_ / _ \| '__/ _` |/ _ \
 |  __/| |>  <  __/ |  _| (_) | | | (_| |  __/
 |_|   |_/_/\_\___|_|_|  \___/|_|  \__, |\___|
                                    |___/
  Kernel: Linux 6.12.63+deb13u1 aarch64
  IP:     192.168.1.xx
  User:   danielsson
  Uptime: 0:00:47

  Escribe  robomenu  para abrir el menú interactivo.
```

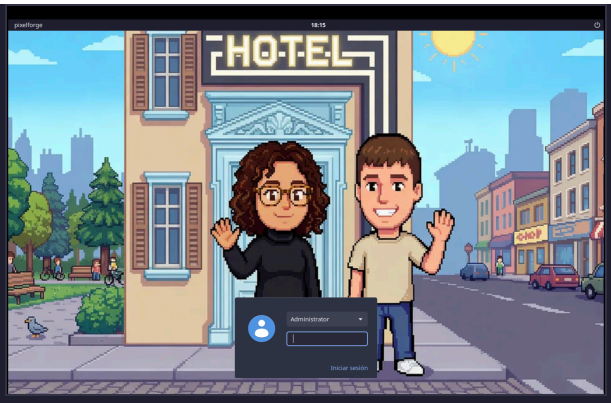
10 — SISTEMA FINAL

# Sistema final: cómo quedó todo

La v20 es Debian 13 Trixie ARM64 con **1660 paquetes** instalados. Arranca en XFCE 4.18 con autologin para `danielsson`. Tema Catppuccin Mocha Blue, wallpaper pixel art del edificio UDIT y menú interactivo `robomenu`.

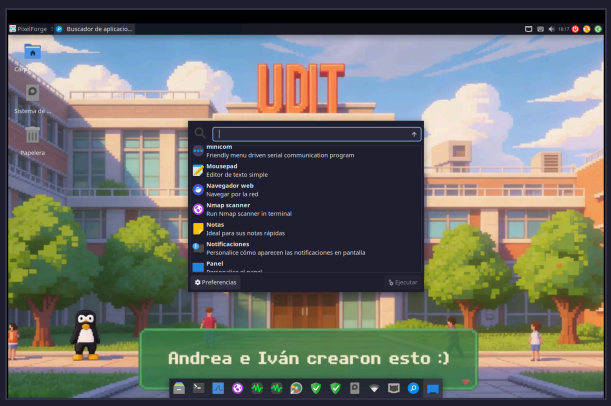
## // EL ARRANQUE PASO A PASO

GRUB — 4 opciones: PixelForge Live (default, seleccionada en 5 segundos), modo seguro, Terminal (arranque directo en TTY, sin escritorio), Debug

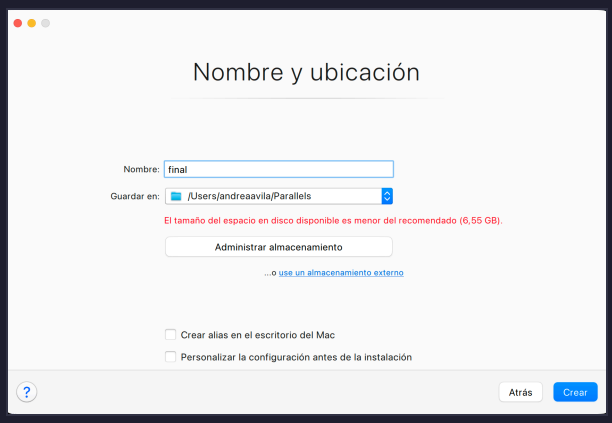



Login LightDM — fondo del hotel con los avatares de Andrea e Iván. El sistema hace autologin automático para danielsson si no se interactúa en 5 segundos.

Escritorio XFCE con el wallpaper del edificio UDIT en pixel art. El texto "Andrea e Iván crearon esto :)" está en el centro del fondo.




Buscador de aplicaciones — minicom, Nmap Scanner, Mousepad y el resto de herramientas instaladas y accesibles desde la interfaz gráfica.




Parallels Desktop — diálogo "Nombre y ubicación" al crear una nueva VM a partir de la ISO v20. Tipo de OS: Other Linux (ARM64).

## // USUARIOS DEL SISTEMA

| USUARIO | CONTRASEÑA | FUNCIÓN | GRUPOS |
|---|---|---|---|
| danielsson | root | Principal — autologin XFCE | sudo, wireshark, audio, video, netdev, plugdev |
| user | live | Secundario | sudo, wireshark, audio, video |
| admin | admin | Administrador | sudo, wireshark |
| root | root | Superusuario | — |

## // PAQUETES INSTALADOS POR CATEGORÍA

### ANÁLISIS DE RED

wireshark · tshark · nmap · masscan · tcpdump · netcat-openbsd · arp-scan · whois · traceroute · iperf3 · ethtool

**SEGURIDAD DEFENSIVA**

lynis · fail2ban · ufw · chkrootkit · rkhunter · clamav · aide · apparmor

**SEGURIDAD OFENSIVA**

hydra · john · hashcat · aircrack-ng · gobuster — nikto: no disponible en Debian 13 Trixie

**DIAGNÓSTICO DEL SISTEMA**

htop · btop · glances · lsof · iotop · sysstat · strace · fastfetch

**GESTIÓN DE DISCOS**

gparted · smartmontools · testdisk · photorec · ddrescue · cryptsetup · parted · gdisk

**RECUPERACIÓN**

chntpw · testdisk · ddrescue · safecopy

**WIFI**

iw · wavemon · wireless-tools · NetworkManager

**ROBÓTICA / HARDWARE**

avrdude · minicom · screen · picocom · python3-pip · esptool (pip)

**ESCRITORIO**

XFCE 4.18 · Catppuccin Mocha Blue · LightDM · Firefox ESR · Thunar · Mousepad · Ristretto

## // PAQUETES QUE NO ESTÁN Y POR QUÉ

| PAQUETE | MOTIVO |
| --- | --- |
| `nikto` | Eliminado de Debian 13 Trixie. Instalable post-boot: `git clone https://github.com/sullo/nikto` |
| `neofetch` | Upstream archivado desde 2024. Sustituido por `fastfetch`. |
| `metasploit-framework` | Requiere instalador Rapid7, ~2 GB adicionales. No empaquetado en Debian. |
| `ros-jazzy-ros-base` | Solo binarios para Ubuntu 24.04 amd64. Build desde fuente: >4h y 5 GB. |
| `kicad` | +1 GB instalado. Incompatible con el objetivo de mantener la ISO manejable. |

11 — INSTRUCCIONES

# Cómo probar la ISO

### // EN PARALLELS DESKTOP (APPLE SILICON) — LO QUE NOSOTROS USAMOS

1. Abrir Parallels Desktop → botón **+** → *Install Windows or another OS from a DVD or image file*
2. Seleccionar `debian-udit-pixelart-arm64-v20-final.iso`
3. Tipo de OS: **Other Linux (ARM64)**
4. Configuración recomendada: 4 núcleos, 4 GB de RAM
5. Arrancar. GRUB con 4 opciones — la primera (PixelForge Live) es el default.
6. Login automático como `danielsson` después de unos 30 segundos.
7. Para abrir el menú interactivo: terminal → `robomenu`

### // EN QEMU

```
./scripts/run_qemu.sh debian-udit-pixelart-arm64-v20-final.iso 2048
```

### // VERIFICAR LA INTEGRIDAD DEL ARCHIVO ISO

```
echo "3f8a04a8f1fe6ae5e9962cf555108c669fc7b95f2a28f1aaa97148ef384e612f  \
  debian-udit-pixelart-arm64-v20-final.iso" | sha256sum -c
# Debe responder: debian-udit-pixelart-arm64-v20-final.iso: OK
```

### // DESCARGAR LA ISO

La ISO no está en el repositorio de git (demasiado grande). Está disponible como release en GitHub:

```
https://github.com/Andreavilaro0/udit-pixelforge/releases/tag/v20
```

---

12 — CONCLUSIONES

# Lecciones aprendidas

1. **Cubic no funciona para ISOs ARM64 en Parallels Desktop.** El efi.img de 32 MB fuerza Ldsiz=0 en el catálogo El-Torito. Parallels rechaza la ISO. QEMU no. Esta diferencia ocultó el problema durante días.
2. **-isohybrid-apm-hfsplus es obligatorio para Parallels en Apple Silicon.** Añade un Apple Partition Map al ISO. No aparece en ninguna guía de xorriso — solo en el archivo .disk/mkisofs que Debian guarda dentro de sus propias ISOs.
3. **libre.debian.net es la fuente correcta para Debian ARM64 live.** La web oficial debian.org no publica live builds para ARM64 — solo ISOs de instalación.
4. **apt install es una transacción atómica.** Un paquete inexistente cancela toda la lista. Para listas de instalación con paquetes potencialmente ausentes, usar bucle por paquete con set +e.
5. **El filtro -Xbcj arm de mksquashfs es para ARM 32-bit.** En ARM64 el squashfs resultante pasa unsquashfs pero el driver del kernel 6.12 lo rechaza durante el arranque. Siempre verificar el formato con unsquashfs -s antes de recompilar.
6. **apt autoremove en un chroot de live es peligroso.** Puede eliminar paquetes marcados como "Recommended" — como xfce4-session — que son necesarios para el escritorio.
7. **/var/lib/apt/lists/ se borra con apt-get clean.** Si planeas instalar más cosas después, siempre ejecutar apt-get update al inicio de cada sesión de chroot.
8. **La carpeta compartida de Parallels es la única forma segura de mover ISOs al Mac.** OneDrive y otros sync clouds corrompen archivos binarios grandes.
9. **Debian instala lolcat en /usr/games/, no en /usr/bin/.** Scripts que dependen de herramientas "fun" deben buscar en múltiples rutas.
10. **tput cols puede devolver un valor incorrecto.** Para scripts que dependen del ancho de terminal, usar las tres fuentes ($COLUMNS, stty size, tput cols) y tomar el menor.

```
UDIT PixelForge v20 · Andrea Ávila Romero · Iván Herrera González
Sistemas Operativos Avanzados · UDIT · 2025–2026
```

```
SHA-256 3f8a04a8f1fe6ae5e9962cf555108c669fc7b95f2a28f1aaa97148ef384e612f
```